

POLITECHNIKA POZNAŃSKA
WYDZIAŁ AUTOMATYKI, ROBOTYKI I
ELEKTROTECHNIKI

Instytut Robotyki i Inteligencji Maszynowej
Zakład Automatyki i Optymalizacji



Jan Andrzejewski
Mateusz Banaszak
Piotr Bednarek

PROJEKT I BUDOWA UKŁADU REGULACJI
DLA UKŁADU BALL ON BEAM

Promotor: dr inż. Jacek Michalski

Pracę przyjmuje i akceptuje

.....
data i podpis opiekuna pracy

Poznań, 2026

Streszczenie

Praca przedstawia projekt i budowę autonomicznego stanowiska do sterowania pozycją kulki na belce (ball and beam) z wykorzystaniem mikrokontrolera STM32F767ZI. Opisano budowę mechaniczną stanowiska, zastosowane peryferia (serwomechanek MG946R, kamera systemu wizyjnego, potencjometr) oraz architekturę oprogramowania opartą na systemie FreeRTOS. Wyprowadzono model matematyczny układu i zaprojektowano dwa regulatory: klasyczny PID z anti-windup oraz optymalny LQR w przestrzeni stanów. Aplikacja nadzorcza napisana w Pythonie (PySide6) umożliwia wizualizację sygnałów w czasie rzeczywistym, zapis danych do pliku CSV oraz zdalne strojenie regulatora. Przeprowadzono weryfikację eksperymentalną i potwierdzono zgodność zachowania fizycznego systemu z wyznaczonym modelem.

Słowa kluczowe: *regulacja automatyczna, kulka na belce, STM32, PID, LQR, FreeRTOS, systemy wizyjne.*

Abstract

Title: *Design and Construction of a Control System for the Ball and Beam System.*

This thesis presents the design and construction of an autonomous ball-and-beam control system based on the STM32F767ZI microcontroller. The mechanical setup, hardware peripherals (MG946R servo, computer vision camera, potentiometer), and FreeRTOS-based software architecture are described. A mathematical model of the plant is derived and two controllers are designed: a classical PID with anti-windup and an optimal LQR state-feedback controller. A supervisory desktop application (Python/PySide6) provides real-time signal visualisation, CSV data logging, and remote parameter tuning. Experimental verification confirms that physical system behaves the same as the theoretical model.

Key words: *automatic control, ball and beam, STM32, PID, LQR, FreeRTOS, computer vision.*

Spis treści

1	Wprowadzenie	3
1.1	Cel i zakres pracy	3
1.2	Przegląd literatury	4
1.3	Struktura pracy	4
2	Budowa stanowiska i oprogramowanie	5
2.1	Opis stanowiska laboratoryjnego	5
2.1.1	Wykaz komponentów	6
2.2	Mikrokontroler STM32 i peryferia	6
2.2.1	Mikrokontroler STM32F767ZI	6
2.2.2	Serwomechanek MG946R	7
2.2.3	Kamera systemu wizyjnego	7
2.2.4	Potencjometr	8
2.3	Środowisko programistyczne	8
2.3.1	Firmware STM32	8
2.3.2	Aplikacja desktopowa	8
2.4	Implementacja algorytmu sterowania	9
2.4.1	Pętla sterowania i okres próbkowania	9
2.4.2	Ograniczenie zakresu serwa	9
2.4.3	Protokół telemetry UART z sumą kontrolną CRC-8	10
2.4.4	Zadania FreeRTOS	10
2.5	System wizyjny	11
2.5.1	Konfiguracja kamery i akwizycja obrazu	11
2.5.2	Detekcja kulki metodą progowania w przestrzeni HSV	11
2.5.3	Wyznaczanie pozycji belki na podstawie znaczników ArUco	12
2.5.4	Kalibracja geometryczna i przeliczanie piksel $\rightarrow$ mm	13
2.5.5	Protokół komunikacyjny PC $\rightarrow$ STM32	13
3	Model matematyczny i projekt regulacji	15
3.1	Model matematyczny obiektu regulacji	15
3.1.1	Dynamika serwomechanizmu	15
3.1.2	Dynamika kulki na belce	16
3.1.3	Połączenie mechaniczne i transmitancja całkowita	17
3.2	Identyfikacja parametrów modelu	17
3.2.1	Sygnał PRBS	17

3.2.2	Przyjęte parametry modelu	18
3.3	Projekt regulatora LQR	18
3.3.1	Opis przestrzeni stanów	18
3.3.2	Sterowalność	18
3.3.3	Obserwowalność	19
3.3.4	Regulator LQR	19
3.4	Projekt regulatora PID	19
4	Wyniki eksperymentalne	21
4.1	Metodyka przeprowadzonych badań	21
4.2	Wyniki pomiarów	21
4.3	Analiza i omówienie wyników	21

Wprowadzenie

Układ kulka na belce (*ball and beam*) jest klasycznym laboratoryjnym obiektem sterowania, powszechnie stosowanym w dydaktyce automatyki ze względu na prostą budowę mechaniczną i zarazem nieliniową, niestabilną dynamikę [1]. Zadanie sterowania polega na utrzymaniu toczącego się swobodnie po belce metalowego walca w zadanej pozycji poprzez regulację kąta wychylenia belki napędzanej serwomechanizmem. Pomimo pozornej prostoty obiekt ten wymaga zastosowania metod nowoczesnej teorii sterowania, co czyni go idealną platformą do weryfikacji algorytmów regulacji zarówno klasycznych, jak i optymalnych.

1.1 Cel i zakres pracy

Celem niniejszej pracy jest zaprojektowanie i zbudowanie autonomicznego stanowiska sterowania kulką na belce opartego na mikrokontrolerze STM32F767ZI [2] oraz zaimplementowanie i porównanie dwóch algorytmów regulacji:

- regulatora **PID** z filtrem pochodnej, członem anti-windup i strojeniem w oparciu o odpowiedź skokową i sygnał APRBS,
- regulatora **LQR** (*Linear Quadratic Regulator*) w przestrzeni stanów z wektorem stanu obejmującym pozycję kulki, jej prędkość oraz kąt belki.

Zakres pracy obejmuje:

- (i) identyfikację modelu matematycznego obiektu metodami doświadczalnymi,
- (ii) projektowanie i dobór nastaw regulatorów PID oraz LQR,
- (iii) budowę stanowiska laboratoryjnego z czujnikiem ToF VL53L0X [3] i serwomechanizmem MG90S,
- (iv) implementację algorytmów w środowisku FreeRTOS na platformie STM32,
- (v) opracowanie aplikacji nadzorczej w języku Python (PySide6) z podglądem kamery i detekcją pozycji kulki metodami wizji komputerowej [4],
- (vi) eksperymentalną weryfikację i porównanie obu strategii sterowania.

1.2 Przegląd literatury

Układ kulka na belce jest od dziesięcioleci obiektem intensywnych badań w dziedzinie automatyki i robotyki. Pierwszym krokiem projektowania regulatora jest zawsze wyrowadzenie modelu matematycznego. Bolívar-Vincenty i Beauchamp-Báez [1] wykazali, że równania ruchu otrzymane metodą Newtona i metodą Lagrange’a są tożsame, o ile poprawnie uwzględni się człony wynikające z obrotu układu odniesienia — błąd często popełniany w literaturze. Autorzy wyprowadzają nieliniowe równania stanu, a następnie linearyzują je w otoczeniu punktu równowagi, uzyskując transmitancję i liniowy model przestrzeni stanów, stanowiący punkt wyjścia do syntezy regulatora.

Wśród klasycznych podejść do sterowania układem dominuje regulator PID. Taifour Ali i in. [5] przedstawili implementację dwupętlowego regulatora PID na mikrokontrolerze Arduino z ultradźwiękowym czujnikiem odległości i serwomechanizmem DC. Autorzy potwierdzili skuteczność ręcznego doboru nastaw metodą prób i błędów, uzyskując stabilizację z minimalnymi oscylacjami przy $K_p=4$, $K_i=1$, $K_d=2$. W pracy wykazano, że dołączenie członu całkującego istotnie redukuje uchyb ustalony, a pochodna skraca czas regulacji i tłumi przeregulowania.

Projektowanie optymalnych regulatorów stanu wymaga solidnych podstaw teoretycznych. Ogata [6] oraz Franklin i in. [7] dostarczają kompletnego opisu teorii przestrzeni stanów, analizy sterowalności i obserwowalności oraz syntezy LQR minimalizującego wskaźnik jakości kwadratowej. W niniejszej pracy macierze wag $\mathbf{Q}$ i $\mathbf{R}$ dobrano metodą iteracyjną z pomocą środowiska MATLAB/Simulink, a uzyskane wzmocnienia $\mathbf{K}$ przeniesiono bezpośrednio do firmware’u mikrokontrolera.

Dodatkową funkcją stanowiska jest wizualna weryfikacja pozycji kulki za pomocą kamery. Przegląd metod śledzenia obiektów w wizji komputerowej przeprowadzony przez Kadama i in. [4] wskazuje, że klasyczne podejście oparte na transformacji przestrzeni barw HSV i detekcji okręgów metodą Hougha pozostaje skuteczne dla obiektów o regularnym kształcie i kontrolowanym oświetleniu — warunki spełnione na stanowisku laboratoryjnym. Głębsze metody uczenia maszynowego byłyby uzasadnione w środowiskach o zmiennym tle lub przy wielu obiektach; dla niniejszego zastosowania przyjęto rozwiązanie klasyczne.

1.3 Struktura pracy

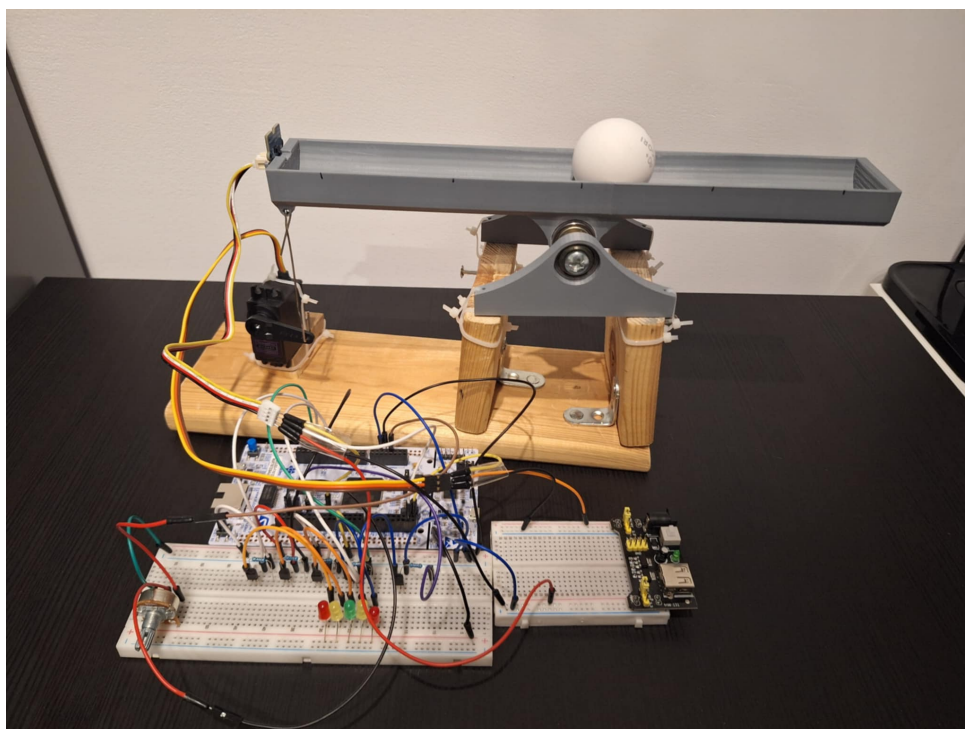
Praca podzielona jest na pięć rozdziałów. Rozdział 2 opisuje budowę mechaniczną i elektryczną stanowiska oraz architekturę oprogramowania — firmware FreeRTOS i aplikację nadzorczą Python. Rozdział 3 zawiera wyprowadzenie modelu matematycznego obiektu metodą Lagrange’a, linearyzację w punkcie pracy oraz projekt regulatorów PID i LQR. Rozdział 4 prezentuje wyniki eksperymentów identyfikacyjnych i regulacyjnych, w tym porównanie charakterystyk skokowych obu strategii sterowania. Rozdział ?? podsumowuje pracę i wskazuje kierunki dalszych badań.

Budowa stanowiska i oprogramowanie

Niniejszy rozdział opisuje fizyczną budowę stanowiska laboratoryjnego, zastosowane komponenty elektroniczne, środowisko programistyczne oraz architekturę oprogramowania sterującego — firmware mikrokontrolera i aplikację nadzorczą.

2.1 Opis stanowiska laboratoryjnego

Stanowisko składa się z wydrukowanej w technologii 3D pochylonej belki, po której swobodnie toczy się kulka (pileczka ping pongowa). Kąt nachylenia belki kontrolowany jest przez serwomechanek połączony z belką za pomocą dźwigni. Pozycja kulki mierzona jest bezdotykowo za pomocą zaimplementowanego systemu wizyjnego. Mikrokontroler STM32 odczytuje pomiar z czujnika, oblicza sygnał sterujący i generuje odpowiedni sygnał PWM dla serwa.



Rysunek 2.1: Stanowisko laboratoryjne układu kulka na belce

2.1.1 Wykaz komponentów

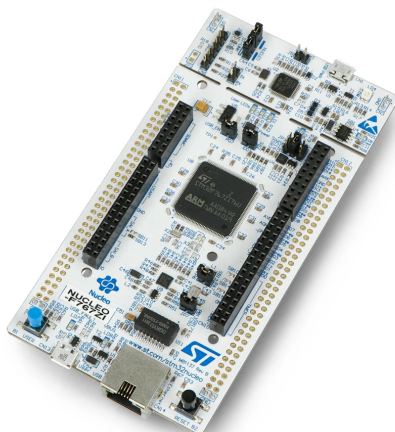
Tabela 2.1: Komponenty sprzętowe stanowiska

Element	Model / opis	Interfejs
Mikrokontroler	STM32 Nucleo F767ZI	—
Serwomechanek	TowerPro MG946R	PWM 50 Hz
Kamera USB	rozdzielczość 640×480, 30 FPS	USB
Potencjometr	liniowy	ADC (PA0)
Zasilacz	5 V DC	—

2.2 Mikrokontroler STM32 i peryferia

2.2.1 Mikrokontroler STM32F767ZI

Głównym procesorem stanowiska jest mikrokontroler STM32F767ZI [2] zamontowany na płytce ewaluacyjnej Nucleo-F767ZI. Układ oparty jest na rdzeniu ARM Cortex-M7 taktowanym z częstotliwością 216 MHz i wyposażony jest w bogatą gamę peryferiów: kilka interfejsów I²C i UART, timery sprzętowe, przetwornik ADC oraz obsługę sygnałów PWM.



Rysunek 2.2: Płytką ewaluacyjna STM32 Nucleo-F767ZI

2.2.2 Serwomechanek MG946R

Do sterowania kątem belki zastosowano serwomechanek TowerPro MG946R. Serwo przyjmuje sygnał PWM o częstotliwości 50 Hz i czasie wypełnienia od 1 ms (pełne wychylenie w jedną stronę) do 2 ms (pełne wychylenie w drugą stronę). Zakres roboczy ograniczony jest w oprogramowaniu do 60–140° w celu zabezpieczenia mechaniki.



Rysunek 2.3: Serwomechanek TowerPro MG946R

2.2.3 Kamera systemu wizyjnego

Do rejestracji obrazu zastosowano kamerę USB o rozdzielczości 640×480 pikseli pracującą z częstotliwością 30 klatek na sekundę. Kamera umieszczona jest prostopadłe do belki, obejmując całą jej długość wraz z narożnymi znacznikami ArUco. Obraz pozyskiwany jest przez bibliotekę OpenCV za pomocą interfejsu `cv2.VideoCapture`, a aplikacja umożliwia wybór indeksu kamery z listy rozwijanej w panelu wizyjnym.

2.2.4 Potencjometr

Potencjometr liniowy podłączony do wejścia analogowego PA0 (ADC1_CH0) umożliwia ręczne zadawanie wartości referencyjnej pozycji kulki w trybie analogowym. Przetwornik ADC skonfigurowany jest z rozdzielczością 12-bitową (0–4095), co odpowiada zakresowi pozycji 0–250 mm.

2.3 Środowisko programistyczne

2.3.1 Firmware STM32

Oprogramowanie mikrokontrolera zostało opracowane w środowisku STM32CubeIDE (Eclipse CDT ze zintegrowanym kompilatorem ARM GCC). Konfigurację peryferiów (timery, I²C, UART, ADC, PWM) wygenerowano za pomocą narzędzia STM32CubeMX. Kod źródłowy podzielono na osobne moduły C zgodnie ze standardem Doxygen:

Tabela 2.2: Moduły firmware

Moduł	Opis
main.c/h	Główna aplikacja, konfiguracja peryferiów
servo_pid.c/h	Regulator PID z anti-windup
lqr.c/h	Regulator LQR (przestrzeń stanów)
filters.c/h	Filtry cyfrowe (EMA, mediana, 1-Euro)
crc8.c/h	Suma kontrolna CRC-8
calibration.c/h	Kalibracja czujnika VL53L0X
vl53l0x.c/h	Sterownik czujnika (I ² C)

System operacyjny czasu rzeczywistego FreeRTOS zarządza wykonaniem dwóch wątków (tasków):

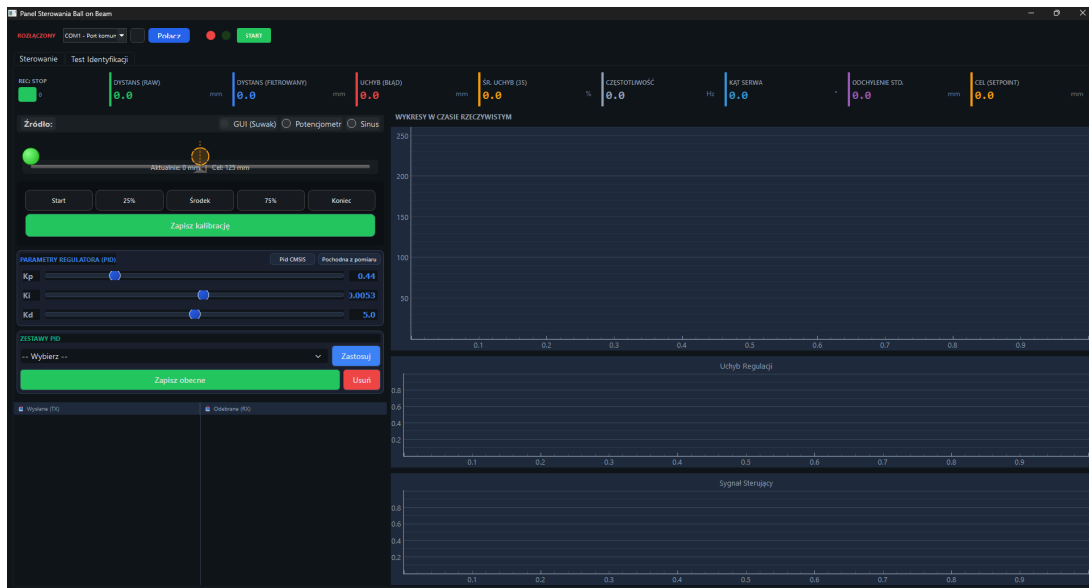
- `ControlTask` (priorytet wysoki) — pętla sterowania;
- `DefaultTask` (priorytet normalny) — obsługa komend UART.

2.3.2 Aplikacja desktopowa

Aplikacja nadzorcza napisana jest w języku Python z wykorzystaniem frameworka PySide6 (Qt 6). Zapewnia ona:

- połączenie z mikrokontrolerem przez port szeregowy (pyserial),
- wizualizację sygnałów w czasie rzeczywistym (pyqtgraph),
- zadawanie wartości referencyjnej i nastaw regulatorów,
- nagrywanie danych i eksport do pliku CSV,

- podgląd kamery z detekcją pozycji kulki (OpenCV).



Rysunek 2.4: Okno aplikacji desktopowej (PySide6)

2.4 Implementacja algorytmu sterowania

2.4.1 Pętla sterowania i okres próbkowania

Algorytm sterowania realizowany jest przez `ControlTask` z ustalonym okresem próbkowania 30 ms???TODO wymuszonym przez funkcję `osDelay()`:

```

1 #define PID_DT_MS 30
2
3 for (;;) {
4     distance = readRangeContinuousMillimeters(&distanceStr);
5
6     // obliczenia regulatora ...
7
8     SetServoAngle(pid_output);
9     osDelay(PID_DT_MS); // stały okres 30 ms
10 }

```

Listing 2.1: Stały okres próbkowania w pętli sterowania (main.c)

2.4.2 Ograniczenie zakresu serwa

Wyjście regulatora jest nasycane do bezpiecznego zakresu kąta serwa:

```

1 #define SERVO_MIN_LIMIT 60.0f
2 #define SERVO_MAX_LIMIT 140.0f
3

```

```
4 if (pid_angle < SERVO_MIN_LIMIT) pid_angle = SERVO_MIN_LIMIT;
5 if (pid_angle > SERVO_MAX_LIMIT) pid_angle = SERVO_MAX_LIMIT;
6 SetServoAngle(pid_angle);
```

Listing 2.2: Saturacja wyjścia i ograniczenia serwa (main.c)

2.4.3 Protokół telemetry UART z sumą kontrolną CRC-8

Dane pomiarowe przesyłane są przez UART 3 (115200 baud) w formacie tekstowym z sumą kontrolną CRC-8 (wielomian 0x07). Format ramki:

D:%d;Z:%d;A:%d;F:%d;E:%d;V:%d;S:%d;C:%02X\r\n

gdzie kolejne pola oznaczają: surowy dystans, setpoint, kąt serwa, dystans filtrowany, uchyb, średni uchyb, status czujnika i CRC.

```
1 uint8_t CalculateCRC8(const char *data, int len) {
2     uint8_t crc = 0x00;
3     for (int i = 0; i < len; i++) {
4         crc ^= data[i];
5         for (uint8_t j = 0; j < 8; j++) {
6             if (crc & 0x80)
7                 crc = (crc << 1) ^ 0x07;
8             else
9                 crc <<= 1;
10        }
11    }
12    return crc;
13 }
```

Listing 2.3: Implementacja CRC-8 (crc8.c)

2.4.4 Zadania FreeRTOS

System operacyjny czasu rzeczywistego tworzy dwa wątki:

```
1 osThreadDef(defaultTask, StartDefaultTask,
2             osPriorityNormal, 0, 256);
3 defaultTaskHandle = osThreadCreate(osThread(defaultTask), NULL);
4
5 osThreadDef(ControlTask, StartControlTask,
6             osPriorityHigh, 0, 1024);
7 ControlTaskHandle = osThreadCreate(osThread(ControlTask), NULL);
8
9 osKernelStart();
```

Listing 2.4: Tworzenie zadań FreeRTOS (main.c)

2.5 System wizyjny

Stanowisko wyposażone jest w podsystem wizji komputerowej realizujący bezdotykowy pomiar pozycji kulki oraz kąta nachylenia belki. Akwizycja i przetwarzanie obrazu realizowane są po stronie aplikacji desktopowej (Python / OpenCV), a wyniki przesyłane są do mikrokontrolera przez interfejs UART.

tutaj jakis screen z apki @piotrk

2.5.1 Konfiguracja kamery i akwizycja obrazu

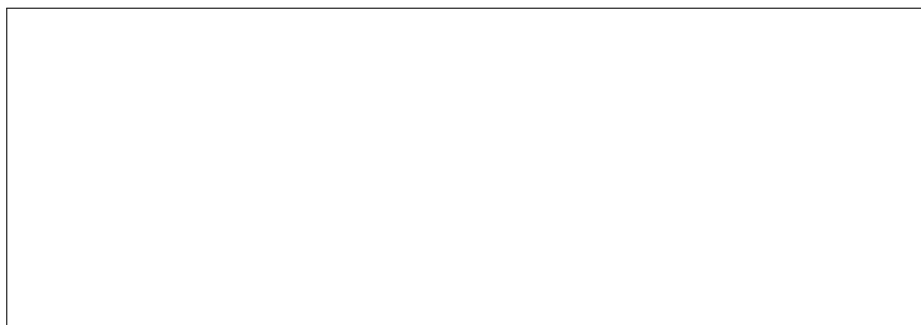
Akwizycja obrazu realizowana jest za pomocą funkcji `cv2.VideoCapture` z biblioteki OpenCV. Kamera konfigurowana jest na stałą rozdzielczość 640×480 pikseli i częstotliwość 30 FPS z minimalnym buforem klatek (`CAP_PROP_BUFFERSIZE = 1`) w celu minimalizacji opóźnienia pomiarowego. Aplikacja obsługuje automatyczne wykrywanie dostępnych kamer (tryb auto) lub ręczny wybór indeksu urządzenia przez użytkownika. Przetwarzanie kolejnych klatek wyzwala się przez timer Qt uruchamiany z jak największą częstotliwością.

2.5.2 Detekcja kulki metodą progowania w przestrzeni HSV

Kulka (piłeczka ping-pongowa) wykrywana jest na podstawie koloru w przestrzeni barw HSV (*Hue, Saturation, Value*). Przetwarzanie każdej klatki przebiega w następujących krokach:

1. **Rozmycie gaussowskie** — redukcja szumów; jądro 21×21 px.
2. **Konwersja BGR $\rightarrow$ HSV** (`cv2.cvtColor`).
3. **Progowanie zakresu barwy** (`cv2.inRange`): $H \in [8, 64]$, $S \in [27, 141]$, $V \in [121, 255]$.
4. **Operacje morfologiczne** — domknięcie (`MORPH_CLOSE`) i otwarcie (`MORPH_OPEN`) na jądrze eliptycznym 5×5 px; usuwają drobne artefakty i wypełniają luki w obrysie kulki. TUTAJ COS DOPSIAC @piotrk
5. **Wykrywanie konturów** (`cv2.findContours`) — wybierany jest kontur o największym polu; jeśli przekracza próg obszaru minimalnego (1480 px^2), wyznaczane jest minimalne koło okalające (`cv2.minEnclosingCircle`).

W celu zmniejszenia czasu obliczeniowego stosowany jest dynamiczny obszar zainteresowania (ROI): w każdej klatce przeszukiwane jest okno ± 100 px wokół ostatnio wyznaczonej pozycji kulki, a pełny obraz skanowany jest tylko przy utracie śledzenia. Podgląd maski HSV oraz obrazu wynikowego z zaznaczoną kulką przedstawiony jest na rys. 2.5.



Rysunek 2.5: Widok zakładki detekcji kulki: obraz oryginalny (lewy górny), maska HSV (prawy górny) oraz wynik z zaznaczonym okręgiem (dolny)

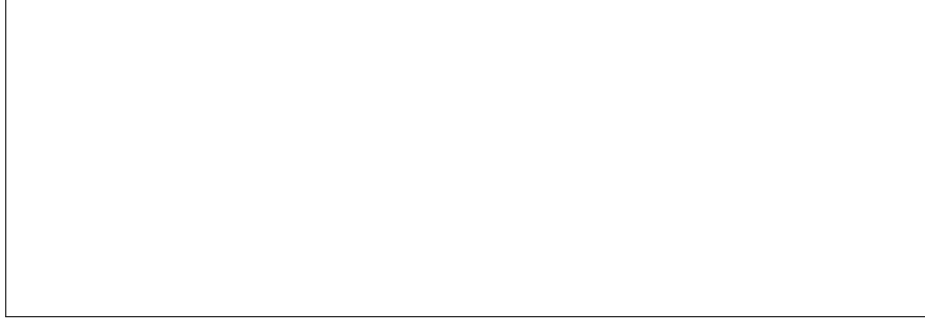
@piotrku

2.5.3 Wyznaczanie pozycji belki na podstawie znaczników ArUco

Na ramie stanowiska przyklejone są cztery kwadratowe znaczniki ze słownika `DICT_4X4_50`: ID 0 (prawy koniec belki), ID 1 i ID 3 (punkty referencyjne ramy) oraz ID 2 (lewy koniec belki). Detekcja znaczników realizowana jest przez moduł `cv2.aruco.ArucoDetector` z następującymi ustawieniami poprawiającymi wykrywanie małych i zaciemnionych markerów:

- preprocessing CLAHE (adaptacyjne wyrównanie histogramu lokalnego) na każdym wycinku ROI markera — lepsze radzenie sobie z nierównomiernym oświetleniem;
- skalowanie podpikselowe ($\times 3$) i wyostrażanie jądrem 3×3 przed detekcją; @piotrku co to jest skalowanie podpikselowe xd???
- precyzyjne dopasowanie narożników metodą AprilTag (`CORNER_REFINE_APRILTAG`);
- rozluźnione progi geometryczne (dopuszczalna deformacja markera, minimalna długość krawędzi 0,1 % szerokości obrazu).

Każdy ze znaczników śledzony jest w osobnym dynamicznym ROI o marginesie 30 px. Mechanizm *position holding* pozwala utrzymać ostatnio wyznaczoną pozycję markera przez maksymalnie 15 klatek przy chwilowej utracie widoczności, co zapobiega skokom w pomiarze kąta. Rozmieszczenie znaczników na stanowisku przedstawione jest na rys. 2.6.



Rysunek 2.6: Stanowisko z nałożonymi wykrytymi znacznikami ArUco: ID 2 (lewy koniec), ID 0 (prawy koniec), ID 1 i ID 3 (punkty referencyjne ramy)

@piotrku

2.5.4 Kalibracja geometryczna i przeliczanie piksel $\rightarrow$ mm

Pozycja kulki wyznaczana jest jako rzut prostokątny jej środka na oś belki (odcinek łączący środki znaczników ID 2 i ID 0):

$$t = \frac{(\mathbf{p} - \mathbf{A}) \cdot (\mathbf{B} - \mathbf{A})}{\|\mathbf{B} - \mathbf{A}\|^2}, \quad t \in [0, 1], \quad (2.1)$$

gdzie $\mathbf{p}$ to współrzędne środka kulki w pikselach, $\mathbf{A}$ i $\mathbf{B}$ to odpowiednio środki ID 2 i ID 0.

Parametr t przeliczany jest na milimetry za pomocą kalibracji dwupunktowej: użytkownik umieszcza kulkę kolejno przy lewym i prawym końcu belki, rejestrując wartości $t_{\min} = 0,039$ i $t_{\max} = 0,990$. Wzór konwersji: @piotrku czy te wartości powinny być tutaj wpisane???

$$x_{\text{mm}} = \frac{t - t_{\min}}{t_{\max} - t_{\min}} \cdot L, \quad (2.2)$$

gdzie $L = 250$ mm jest długością aktywnej części belki.

Kąt nachylenia belki wyznaczany jest jako kąt odcinka ID 2–ID 0 względem poziomu obrazu (metoda domyślna) lub względem linii referencyjnej ID 1–ID 3 (metoda alternatywna dostępna z GUI). Parametry kalibracji zapisywane są do pliku `opencv_params.json` i wczytywane automatycznie przy kolejnym uruchomieniu aplikacji.

2.5.5 Protokół komunikacyjny PC $\rightarrow$ STM32

Wyniki pomiaru (pozycja kulki i kąt belki) przesyłane są do mikrokontrolera przez ten sam port szeregowy UART (115200 baud), którym odbywa się telemetria z STM32. Format ramki wizyjnej:

V:%d.%d;B:%d.%d;C:%02X\n

gdzie V to pozycja kulki w mm, B to kąt belki w stopniach, a C to suma kontrolna CRC-8 (wielomian 0x07) obejmująca pola V i B . Ramki wysyłane są z częstotliwością 30 Hz przez dedykowany timer Qt.

Po stronie firmware funkcja `ParseVisionFrame()` (plik `main.c`) odróżnia ramkę wizyjną od standardowych komend sterujących po pierwszym znaku V , waliduje CRC i aktualizuje zmienne globalne `g_vision_ball_pos` i `g_vision_beam_angle`. Jeśli przez ponad 200 ms nie nadejdzie żadna poprawna ramka wizyjna, regulator automatycznie przechodzi w tryb fallback: jako pomiar pozycji używana jest ostatnia poprawna wartość, a kąt belki przyjmuje wartość 0 deg. @piotrk nie wiem czemu tutaj nie robi się stopień

```
1 void ParseVisionFrame(const char *frame) {
2     // ...parsowanie pol V i B...
3     if (crc_found && ball_pos >= 0.0f) {
4         uint8_t crc_calc = CalculateCRC8(frame, data_len);
5         if (crc_calc == crc_received) {
6             g_vision_ball_pos = ball_pos;
7             g_vision_beam_angle = beam_angle;
8             g_vision_data_valid = 1;
9             g_vision_last_update = HAL_GetTick();
10        }
11    }
12 }
```

Listing 2.5: Fragment funkcji `ParseVisionFrame()` (`main.c`) — parsowanie i walidacja CRC

Model matematyczny i projekt regulacji

Rozdział ten przedstawia wyprowadzenie modelu matematycznego układu kulka na belce, metodę identyfikacji jego parametrów oraz projekt regulatorów PID i LQR.

3.1 Model matematyczny obiektu regulacji

Układ składa się z trzech elementów połączonych szeregowo: serwomechanizmu, mechanicznego połączenia dźwigniowego oraz kulki toczącej się po belce.

3.1.1 Dynamika serwomechanizmu

Dynamikę serwomechanizmu w pierwszym przybliżeniu opisuje człon inercyjny pierwszego rzędu:

$$G_{\text{serwo}}(s) = \frac{\Theta_{\text{serwo}}(s)}{\Theta_{\text{ref}}(S)} = \frac{1}{T_{\text{serwo}} s + 1}, \quad (3.1)$$

gdzie T_{serwo} jest stałą czasową serwa.

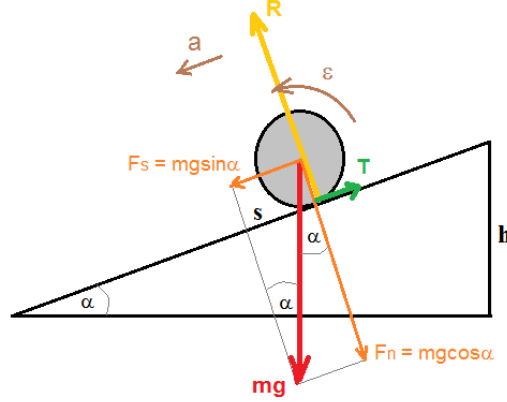
Na podstawie danych katalogowych serwomechanizmu MG946R (0,20 s/60° przy 4,8 V; 0,11 s/60° przy 6 V) interpolując dla napięcia 5 V:

$$V_{5V} \approx 0,20 - \frac{0,09}{1,2} \cdot 0,2 \approx 0,185 \text{ s/60}^\circ. \quad (3.2)$$

Przyjęto stałą czasową $T_{\text{serwo}} = 0,095 \text{ s}$.

3.1.2 Dynamika kulki na belce

Równanie ruchu — II zasada dynamiki



Rysunek 3.1: Rozkład sił działających na kulkę na równi pochyłej

Dla kulki toczącej się bez poślizgu po belce pochyłonej pod kątem θ równanie ruchu postępowego wynosi:

$$m\ddot{x} = m g \sin \theta - T, \quad (3.3)$$

a równanie momentu obrotowego wokół osi kulki:

$$T \cdot R = J \ddot{\alpha}, \quad (3.4)$$

gdzie T — siła tarcia statycznego, R — promień kulki, J — moment bezwładności, $\ddot{\alpha}$ — przyspieszenie kątowe kulki.

Warunek toczenia bez poślizgu

Z warunku $\ddot{x} = \ddot{\alpha} \cdot R$ eliminuje się siłę tarcia:

$$m\ddot{x} + \frac{J}{R^2}\ddot{x} = m g \sin \theta \quad \Rightarrow \quad \ddot{x} = \frac{g \sin \theta}{1 + J/(mR^2)}. \quad (3.5)$$

Dla małych kątów $\sin \theta \approx \theta$, a dla walcowej kulki $J = \frac{2}{3}mR^2$, stąd:

$$\ddot{x} = \underbrace{\frac{g}{1 + 2/3}}_{c_{\text{ball}}} \cdot \theta = 0,6 g \cdot \theta. \quad (3.6)$$

Stała dynamiki kulki wynosi:

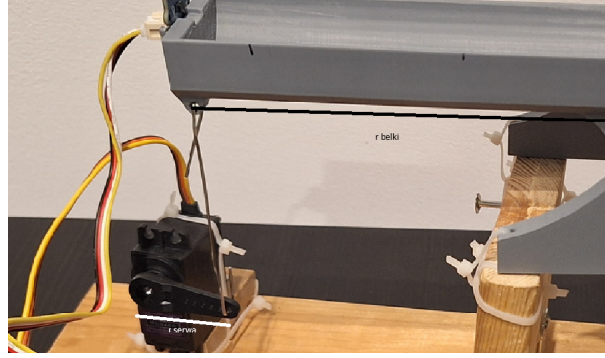
$$c_{\text{ball}} = 0,6 \times 9,81 = 5,886 \text{ m/s}^2. \quad (3.7)$$

Transmitancja kulki na belce

Po zastosowaniu transformaty Laplace'a (zerowe warunki początkowe):

$$G_{\text{ball}}(s) = \frac{X(s)}{\Theta_{\text{beam}}(s)} = \frac{c_{\text{ball}}}{s^2}. \quad (3.8)$$

3.1.3 Połączenie mechaniczne i transmitancja całkowita



Rysunek 3.2: Schemat mechanicznego połączenia serwa z belką (dźwignia)

Kąty serwa i belki powiązane są przez przekładnię mechaniczną:

$$\theta_{\text{beam}} = \frac{r_{\text{serwo}}}{r_{\text{belka}}} \cdot \theta_{\text{serwo}} = k_{\text{mech}} \cdot \theta_{\text{serwo}}, \quad (3.9)$$

gdzie $r_{\text{serwo}} = 3 \text{ cm}$, $r_{\text{belka}} = 15 \text{ cm}$, zatem $k_{\text{mech}} = 0,2$.

Całkowita transmitancja układu (od sygnału sterującego $U(s)$ do pozycji kulki $X(s)$):

$$G_{\text{total}}(s) = k_{\text{mech}} \cdot G_{\text{serwo}}(s) \cdot G_{\text{ball}}(s) = \frac{k_{\text{mech}} \cdot c_{\text{ball}}}{s^2(T_{\text{serwo}} s + 1)}. \quad (3.10)$$

Podstawiając wartości liczbowe:

$$G_{\text{total}}(s) = \frac{0,2 \cdot 5,886}{0,095 s^3 + s^2} = \frac{1,177}{0,095 s^3 + s^2}. \quad (3.11)$$

3.2 Identyfikacja parametrów modelu

3.2.1 Sygnał PRBS

Do identyfikacji parametrów serwa zastosowano sygnał PRBS (*Pseudo Random Binary Sequence*) — deterministyczną sekwencję dwustanową o właściwościach szumopodobnych. PRBS zapewnia szerokie widmo częstotliwościowe i ograniczoną amplitudę, co jest istotne ze względu na stabilność mechaniki stanowiska.

Ze względu na ograniczoną możliwość doboru parametrów PRBS (długości sekwencji i czasu trwania bitu) pokrywających pełną dynamikę obiektu, wyniki identyfikacji oparto ostatecznie na odpowiedzi skokowej i danych katalogowych serwomechanizmu.

3.2.2 Przyjęte parametry modelu

Tabela 3.1: Parametry numeryczne modelu

Parametr	Wartość	Źródło
T_{serwo}	0,095 s	karta katalogowa, interpolacja
c_{ball}	5,886 m/s ²	wyprowadzenie analityczne
k_{mech}	0,20	pomiar geometryczny (3 cm/15 cm)

TODO dokonczyc te sekcje!!!

3.3 Projekt regulatora LQR

3.3.1 Opis przestrzeni stanów

Jako wektor stanu przyjęto trzy fizycznie mierzalne wielkości:

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \end{bmatrix}, \quad (3.12)$$

gdzie x — pozycja kulki, $\dot{x}$ — prędkość kulki, θ — kąt serwa (belki).

Wyprowadzając równania różniczkowe z bloków transmitancyjnych, otrzymuje się model przestrzeni stanów w postaci $\dot{\mathbf{x}} = A\mathbf{x} + Bu$, $y = C\mathbf{x}$:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & c_{\text{ball}} \\ 0 & 0 & -1/T_{\text{serwo}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1/T_{\text{serwo}} \end{bmatrix} u, \quad (3.13)$$

$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \mathbf{x}. \quad (3.14)$$

Podstawiając wartości liczbowe ($c_{\text{ball}} = 5,886$, $T_{\text{serwo}} = 0,095$):

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 5,886 \\ 0 & 0 & -10,526 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 10,526 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}. \quad (3.15)$$

3.3.2 Sterowalność

Macierz sterowalności $\mathcal{C} = [B \ AB \ A^2B]$:

$$\mathcal{C} = \begin{bmatrix} 0 & 0 & 12,389 \\ 0 & 12,389 & -110,796 \\ 10,526 & -110,796 & 1,166 \times 10^3 \end{bmatrix}. \quad (3.16)$$

Wyznacznik:

$$\det(\mathcal{C}) = 12,389 \times (-130,380) \approx -1614 \neq 0. \quad (3.17)$$

Macierz ma pełny rząd 3, zatem układ jest **w pełni sterowalny** — możliwe jest zaprojektowanie efektywnego regulatora stanu (LQR).

3.3.3 Obserwowalność

Macierz obserwowalności $\mathcal{O} = [C; CA; CA^2]^T$:

$$\mathcal{O} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1,177 \end{bmatrix}, \quad \det(\mathcal{O}) = 1,177 \neq 0. \quad (3.18)$$

Rząd wynosi 3, układ jest **w pełni obserwowalny** — wszystkie stany można zrekonstruować z pomiaru pozycji kulki x .

3.3.4 Regulator LQR

Regulator LQR (*Linear Quadratic Regulator*) minimalizuje wskaźnik jakości kwadratowej:

$$J = \int_0^\infty (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt, \quad (3.19)$$

gdzie macierze wag Q (penalizacja stanów) i R (penalizacja wejścia) dobrano metodą iteracyjną w środowisku MATLAB/Simulink. Uzyskany wektor wzmocnień $\mathbf{K}$ przeniesiono bezpośrednio do modułu `lqr.c` firmware. Sygnał sterujący wyznaczany jest jako:

$$u = -\mathbf{K} \mathbf{x} = -[K_1 \ K_2 \ K_3] \begin{bmatrix} x \\ \dot{x} \\ \theta \end{bmatrix}. \quad (3.20)$$

TODO dodać jakieś przykładowe macierze Q i R / skrypt w pythonie gdzie to liczyliśmy

3.4 Projekt regulatora PID

W firmware mikrokontrolera zaimplementowano klasyczny regulator PID z następującymi modyfikacjami:

- **pochodna na pomiarze** — różniczkowanie sygnału wyjścia (nie uchybu), co eliminuje skok pochodnej przy zmianie wartości zadanej;
- **anti-windup** (ograniczenie członu całkującego) — zapobiega nasyceniu członu I przy wysyceniu wyjścia;
- **dyskretyzacja** — metoda prostokątów (Euler przód) z krokiem $T_s = 30$ ms.

TODO uzasadnić dlaczego pochodna na pomiarze i dokonać ten rozdział task @Banaszak !!!!!!!

Nastawy regulatora PID (K_p , K_i , K_d) dobierane są iteracyjnie i przesyłane do mikrokontrolera w czasie rzeczywistym przez aplikację desktopową bez konieczności ponownego programowania układu.

Wyniki eksperymentalne

TODO task @Jan !!!! - akwizycja z stanowiska - simulink z danymi z stanowiska i generowaniem odpowiedzi z modelu -opis

4.1 Metodyka przeprowadzonych badań

4.2 Wyniki pomiarów

4.3 Analiza i omówienie wyników

Bibliografia

- [1] Carlos G. Bolívar-Vincenty and Gerson Beauchamp-Báez. Modelling the ball-and-beam system from newtonian mechanics and from lagrange methods. In *Proceedings of the 12th Latin American and Caribbean Conference for Engineering and Technology (LACCEI'2014)*, Guayaquil, Ecuador, July 2014. Paper #PE148.
- [2] STMicroelectronics. *STM32F767ZI Reference Manual (RM0410)*, 2020. Rev 4.
- [3] STMicroelectronics. *VL53L0X World's Smallest Time-of-Flight Ranging and Gesture Detection Sensor*, 2018. Datasheet Rev 1.1.
- [4] Pushkar Kadam, Gu Fang, and Ju Jia Zou. Object tracking using computer vision: A review. *Computers*, 13(6):136, 2024.
- [5] A. Taifour Ali, A. M. Ahmed, H. A. Almahdi, Osama A. Taha, and A. Naser aldeen. Design and implementation of ball and beam system using PID controller. *Automatic Control and Information Sciences*, 3(1):1–4, 2017.
- [6] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, Upper Saddle River, NJ, 5 edition, 2010.
- [7] Gene F. Franklin, J. David Powell, and Abbas Emami-Naeini. *Feedback Control of Dynamic Systems*. Pearson, 7 edition, 2014.

Spis rysunków

2.1	Stanowisko laboratoryjne układu kulka na belce	6
2.2	Płytką ewaluacyjną STM32 Nucleo-F767ZI	7
2.3	Serwomechanek TowerPro MG946R	7
2.4	Okno aplikacji desktopowej (PySide6)	9
2.5	Widok zakładki detekcji kulki: obraz oryginalny (lewy górny), maska HSV (prawy górny) oraz wynik z zaznaczonym okręgiem (dolny)	12
2.6	Stanowisko z nałożonymi wykrytymi znacznikami ArUco: ID 2 (lewy koniec), ID 0 (prawy koniec), ID 1 i ID 3 (punkty referencyjne ramy)	13
3.1	Rozkład sił działających na kulkę na równi pochyłej	16
3.2	Schemat mechanicznego połączenia serwa z belką (dźwignia)	17